

ARM v7-M architecture



R. Ferrero

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



Data types

- The following data types are supported:
 - byte: 8 bits
 - halfword: 16 bits
 - word: 32 bits
- Registers are 32-bit wide.
- Instructions are 16 or 32 bits long.

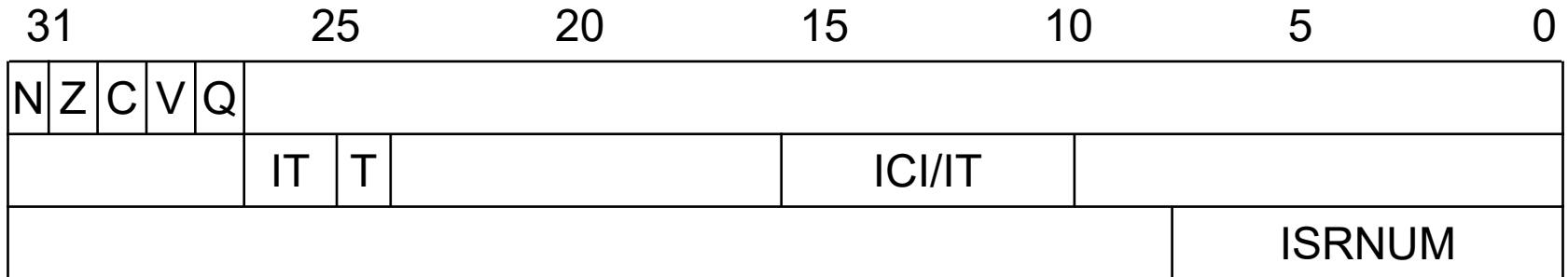
Register	Synonym	Use
r0	a1	general purpose registers
r1	a2	
r2	a3	
r3	a4	
r4	v1	
r5	v2	
r6	v3	
r7	v4	
r8	v5	
r9	v6, SB	
r10	v7	
r11	v8	
r12	IP	
r13	SP, MSP, PSP	stack pointer
r14	LR	link register
r15	PC	program counter

Registers

Special register	Use
xPSR	program status register
PRIMASK	interrupt mask registers
FAULTMASK	
BASEPRI	
CONTROL	control register

Program Status Register

- It can be accessed all at once (PSR) or as a combination of 3 registers:
 - Application Program Status Register (APSR)
 - Execution Program Status Register (EPSR)
 - Interrupt Program Status Register (IPSR)



Application Program Status Register

- It contains:
 - N: Negative result from ALU
 - Z: Zero result from ALU
 - C: ALU operation Carried out
 - V: ALU operation oVerflowed
 - Q: “sticky” flag for saturation arithmetic.

EPSR and IPSR

- The Execution Program Status Register contains:
 - IT: IF-THEN instruction status bits
 - ICI: Interrupt-Continuable Instruction bits
 - T: Thumb bit. It is always 1: clearing this bit will cause a hard fault exception.
- The Interrupt Program Status Register contains an exception number used in exception handling.

Processor modes

- Two operating modes:
 - thread mode: on reset or after completing the exception routine
 - handler mode: when an exception occurs.
- Two access levels:
 - user level: limited access to resources
 - privileged level: access to all resources.
- Handler mode is always privileged.

Control register

- It has 2 bits only:
 - CONTROL[0] changes the access level
 - CONTROL[1] selects which stack is being used (Main Stack Pointer or Process Stack Pointer)
- CONTROL is writable only in privileged level.

	privileged level	user level
handler mode	CONTROL[0] = 0 CONTROL[1] = 0	not allowed
thread mode	CONTROL[0] = 0 CONTROL[1] = 0/1	CONTROL[0] = 1 CONTROL[1] = 0/1

Interrupt vector table

- Each entry manages an exception, interrupt, or other atypical events such as a reset.
- The vector table contains either ARM instructions or addresses in memory.
- In the v7-M architecture, there are addresses.

Interrupt vector table

Exception type	Priority	Position	Vector address
(Top of Stack)		0	0x00000000
Reset	-3	1	0x00000004
NMI	-2	2	0x00000008
Hard fault	-1	3	0x0000000C
Memory management fault	programmable	4	0x00000010
Bus fault	programmable	5	0x00000014
Usage fault	programmable	6	0x00000018
Reserved		7-10	
Supervisor call	programmable	11	0x0000002C
Debug monitor	programmable	12	0x00000030
Reserved		13	
PendSV	programmable	14	0x00000038
SysTick	programmable	15	0x0000003C
External interrupts	programmable	16 - 255	≥0x00000040

Features of ARM Instruction Sets

- Most instructions execute in a single cycle.
- Instructions can be conditionally executed.
- A load/store architecture
 - Data processing instructions act only on registers
 - Two or three operand formats
 - Combined ALU and shifter
 - Memory access instructions with auto-indexing.

Instruction length

- There are 3 kinds of instructions:
 - ARM instructions: 32 bits wide
 - Thumb instructions: 16 bits wide
 - Thumb-2 instructions: 16 or 32 bits wide
- Example: sum of registers
 - 32 bits: `ADD r0, r0, r2`
 - 16 bits: `ADD r0, r2`
- ARM v7-M architecture supports Thumb and Thumb-2 instructions only.

Example: shifting data

```
        AREA |.text|, CODE, READONLY
Reset_Handler PROC
    EXPORT Reset_Handler [WEAK]
    MOV r0, #0x11 ;load initial value
    LSL r1, r0, #1 ;shift 1 bit left
    LSL r2, r1, #1 ;shift 1 bit left
stop B stop ;stop program
    ENDP
```

Example: factorial calculation

```
AREA |.text|, CODE, READONLY
Reset_Handler PROC
    EXPORT Reset_Handler [WEAK]
    MOV r6, #10 ;load 10 into r6
    MOV r7, #1
loop CMP r6, #0
    ITTT GT      ;start of IF-THEN block
    MULGT r7, r6, r7
    SUBGT r6, r6, #1
    BGT loop     ;end of IF-THEN block
stop B stop      ;stop program
    ENDP
```

Example: swapping registers

```
        AREA      |.text|, CODE, READONLY
Reset_Handler PROC
    EXPORT Reset_Handler [WEAK]
    LDR r0,=0xF631024C ;load some data
    LDR r1,=0x17539ABD ;load some data
    EOR r0, r0, r1      ;r0 XOR r1
    EOR r1, r0, r1      ;r1 XOR r0
    EOR r0, r0, r1      ;r0 XOR r1
    stop B stop         ;stop program
    ENDP
```

Keil: installation guide

- Download MDK-Arm (registration required):
<https://www.keil.com/download/product/>
 - Avoid spaces in the installation path
- Download the package LPC1700_DFP, version 2.7.2: https://www.keil.arm.com/packs/lpc1700_dfp-keil/versions/
- Use the Pack Installer to install the package (File -> import...)